

武漢大學

Proposal for ASC24 Preliminary Round

Team: ASC604

Team Member: Xianyi Wei, Xirui Shui, Zhongwei Xie, Han Qiu, Haihang Zou

Supervisor: Zhigao Zheng, Bo Du, Zhengguo Hong

Abstract

This paper provides a comprehensive overview of the performance optimization and evaluation of two prominent benchmarks, High-Performance Linpack (HPL) and High-Performance Conjugate Gradients (HPCG), within a high-performance computing (HPC) environment. The study focuses on the software and hardware environments, test methods, performance optimization methods, and performance evaluation results for both benchmarks.

For HPL, our optimization strategy integrated system-level configurations and algorithmic enhancements, significantly boosting computational efficiency. The performance evaluation revealed promising outcomes, with a marked increase in the floating-point operation rate (FLOP/s), reflecting the success of our tailored optimization approaches.

The HPCG evaluation highlighted suboptimal performance due to the use of reference computational components and insufficient execution time. To address these issues, we propose optimizing core components and increasing test scale and complexity, aiming to meet and surpass official benchmarking standards.

Llama 2 is the latest generation open source big model officially released by Meta AI. It is a computer processing architecture based on distributed memory, with efficient data processing capabilities, flexible scalability, high reliability, low latency, and easy management and maintenance. In this challenge, we are required to make improvements on the inference performance. We end up with the speed results that are 10(GPU acceleration), 3.121(parameter setting), 3(kv cache) throughput compared to the original results, through the optimization of custom operators, Compressed Inference.

We analyse the workflow of Llama 2. Moreover, we design the strategies used to optimize the performance in the process of the inference process. In the end, we evaluate the efficiency of our approaches and come to a conclusion.

For the OpenCAEPoro part, we analyze the source code of OpenCAEPoro and use the Intel Vtune Performance Analyzer to analyze each step and obtain the hot spots of the program, and ultimately focuse on the most time-consuming part of the optimization program, the linear solver.

We combine parallel operations and code optimization to improve efficiency, and ultimately chose to modify the source code of FIM_solver_p_cpr and run 10 processes in parallel on a single node for computation, in order to achieve optimal

efficiency. Compared to running the initial code as a single process on a single node, we have significantly reduced time consumption and increased efficiency by over 4.8×.

Keywords: HPC; HPL; HPCG; Paralle Computing; LLM; OpenCAEPro

Table of Contents

1 Introduction of the university department activities in supercomputing **1**

1.1 Supercomputing-related courses, training, and interest groups 1

1.2 Supercomputing-related research and applications 2

1.3 A brief description of the key achievements in supercomputing 3

2 Team introduction **4**

2.1 Brief description of the team setup 4

2.2 Introduction and the photo of each member 5

2.3 Team's motto or catch-phrase 8

3 Design of HPC system **9**

3.1 Cluster Graph 9

3.2 Design ideas and architectural features 9

3.3 Hardware resources 11

3.4 Software resources 11

3.5 Power Description 12

3.6 Performance Evaluation 13

3.6.1 GPU-Dominated Mode 13

3.6.2 CPU-Dominated Mode 13

4 HPL and HPCG **14**

4.1 HPL 14

4.1.1 Software environment 14

4.1.2 Hardware environment 14

4.1.3 Test methods 15

4.1.4 Performance optimization method 15

4.1.5 Performance evaluation 16

4.1.6 Problem and Solution Analysis 16

4.2 HPCG 17

4.2.1 Software environment 18
4.2.2 Hardware environment 18
4.2.3 Test methods 19
4.2.4 Performance optimization method 19
4.2.5 Performance evaluation 20
4.2.6 Problem and solution analysis 21

5 Optimization for LLM inference 22
5.1 environment 22
5.1.1 Hardware environment 22
5.1.2 Software environment 22
5.2 Test methods 23
5.2.1 Test node usage 23
5.3 Performance Optimization and Performance Improvement Evaluation . 23
5.3.1 GPU acceleration 23
5.3.2 Parameter settings 24
5.3.3 KV cache 25
5.3.4 Performance Improvement evaluation 25

6 The OpenCAEPoro Challenge 26
6.1 Environment 26
6.1.1 Hardware environment 26
6.1.2 Software environment 26
6.2 Performance Optimization and Performance Improvement Evaluation . 27
6.2.1 Program hotspot analysis 27
6.2.2 Parallel computing acceleration 28
6.2.3 Mearsures for acceleration 30
6.2.4 Performance improvement evaluation 31

Conclusion 32

Reference 34

1 Introduction of the university department activities in supercomputing

Wuhan University has a dedicated Supercomputing Center that provides high-performance computing services. The center is located in the East Lake High-tech Development Zone of central China' s Hubei Province.

The Wuhan Supercomputing Center has been connected to China' s national computing network. The first-phase computing power of the center is expected to reach 50 petahash per second, equivalent to 100,000 high-performance computers functioning simultaneously, with a final goal of hitting 200 petahash per second.

The center covers a built-up area of 4,400 square meters and was invested by Hubei Science Technology Investment Group Co., Ltd and Wuhan Industrial Investment Development Group Co., Ltd with a total investment amount of 1 billion yuan (about $14.7 million$).

The Wuhan Supercomputing Center has combined both artificial Intelligence (AI) and high-performance computing (HPC) resources. The Supercomputing Center, together with Wuhan AI Computing Center with computing power of 200 petahash per second, will form diverse computing ability clusters of AI and HPC, empowering industries and stimulating economic momentum.

Both Wuhan Supercomputing Center and Wuhan Artificial Intelligence Computing Center have linked with the national computing network. China' s national computing network serves the country' s strategy of channeling more computing resources from the eastern areas to less developed western regions.

1.1 Supercomputing-related courses, training, and interest groups

Courses: There are many specific courses on supercomputing in WHU, such as the numerical analysis, distributed computing for undergraduates and numerous further specific courses for the graduates . Wuhan University' s Supercomputing

Center provides high-performance computing services. The Intelligent Computing System Lab (ICSLab) also focuses on distributed computing, which includes high-performance computing.

Training: The Supercomputing Center provides a quick start guide for users, covering topics such as logging in, program compilation, job submission, job viewing, and file transfer. The ICSLab also conducts research on a series of intelligent computing systems, including cloud-edge computing systems, big data platforms, and distributed deep learning architectures.

Interest Groups: Not only many fans clubs woking on HPC like Linux Club, Apple Club are founded by students with passions, but also the university' s Supercomputing Center and ICSLab could serve as hubs for students and researchers interested in this field. Additionally, Wuhan University has a collaboration agreement with the High-Performance Computing Center Stuttgart (HLRS) for technology and training projects.

1.2 Supercomputing-related research and applications

Wuhan University' s Intelligent Computing System Lab (ICSLab) and Supercomputing Center are actively involved in supercomputing-related research and applications.

Research: The ICSLab' s main research field is distributed computing, which includes high-performance computing, energy-efficient computing, and scalable computing1. Their work focuses on a series of intelligent computing systems, including cloud-edge computing systems, big data platforms, distributed deep learning architectures, federated learning, graph computing, heterogeneous memory management, and energy-efficient computing platforms1. They have made significant achievements in these areas, with their research being published in prestigious journals and conferences.

Applications: The Supercomputing Center provides high-performance computing services. The numerical calculations for various research papers have been done on the supercomputing system in the Supercomputing Center of Wuhan University.

The ICSLab has developed systems for real-time video analytics, IoT gateways, and more.

1.3 A brief description of the key achievements in supercomputing

Wuhan University's Intelligent Computing System Lab (ICSLab) has made significant achievements in the field of supercomputing:

1. Spatio-temporal Big Data Analytics Ecosystem: They have conducted a survey on the resource management, processing platform, and applications of the Spatio-temporal Big Data Analytics Ecosystem.

2. Edge-side Real-time Video Analytics System: The ICSLab has developed an edge-side real-time video analytics system with dual computing resource control.

3. Hierarchical Pipeline Parallelism: They have developed a redundancy-free high-performance dynamic Graph Neural Network (GNN) training system with hierarchical pipeline parallelism.

4. Memory Efficient Mixture of Experts (MoE) for Pre-trained Models: The ICSLab has developed MPipeMoE, a memory-efficient MoE for pre-trained models with adaptive pipeline parallelism.

5. Accelerating DNN Inference on the Edge through Layer Partitioning: They have developed DNN Surgery, a system for accelerating DNN inference on the edge through layer partitioning.

6. Decentralized Model Aggregation for Scalable Federated Learning: They have developed Spread, a system for decentralized model aggregation for scalable federated learning.

2 Team introduction

2.1 Brief description of the team setup

Our Wuhan University team, led by Dr. Zheng Zhigao, is a collaborative force ready to excel in the upcoming competition. With a strong foundation in scientific computing, we navigate diverse domains like physics, chemistry, and biology using parallel computing, numerical simulations, and algorithm development.

Our collective strengths extend to distributed systems and multitask scheduling optimization, showcased through ongoing work on a high-performance storage system. Leveraging techniques like concurrency, parallelism, and snapshot implementation, we design efficient and adaptive solutions.

Additionally, our team's background in competitive programming brings a deep understanding of performance optimization through data structures and algorithms, coupled with strong code implementation capabilities.

With an AI expert specializing in supercomputing, we bring a multidimensional approach to the competition, ready to tackle complex challenges and contribute meaningfully to the evolving landscape of computer science.



Figure. 2.1 The group photo in our AI school

2.2 Introduction and the photo of each member

Our team includes 5 student members from the Hongyi School of Wuhan University (WHU) and an advisor from the School of Computer Science of Wuhan University. All the members are from diverse disciplines, such as mathematics, physics, chemistry, biology, and computer science. Hongyi School implements a dynamic learning management mechanism and adopts individualized training in small classes, and all the Hongyi students will be sent to world-class universities in stages to learn and communicate through joint training programs. We list the brief introduction of each team members as below:



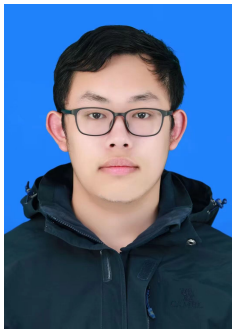
Xianyi Wei is the captain of our team. He enrolled at Wuhan University in Sept. 2021. He has a great passion for computational science and a strong focus on algorithm optimization and resource management. His academic journey has equipped him with a solid foundation in computer science, mathematics, and parallel computing techniques, which are essential for addressing complex problems in the ASC challenge. He got fruitful experience in algorithm optimization and resource allocation from previous research and course projects. He excels in team coordination, having led and contributed to various group projects, which have honed my collaborative and organizational skills. He is excited to participate in the ASC to apply his technical acumen, problem-solving skills, and teamwork abilities to develop innovative solutions that align with the competition's objectives.



Zhongwei Xie is an undergraduate student of computer science, who enrolled in Wuhan University in Sept. 2021. His research interest is focused on the distributed system and the optimization of multitask scheduling. He is working on a high performance storage system that can fully reap the benefits of new hardware such as persistent memory, and multi-core processors. He proposed a new scheme to reduce the write overhead and improve the utilization of the memory system by combining a series of techniques like concurrency and parallelism, kv separation, snapshot, and garbage collection. He has the ability to understand complex problems quickly and come up with innovative solutions. Besides, he is good at cooperating with team members to overcome difficulties together and ensure the smooth progress of the project. This makes me an important communication bridge in the team and promotes the cooperation and coordination of the team.



Haihang Zou is an undergraduate student of computer science at Hongyi School, who enrolled in Wuhan University in Sept. 2021. He has extensive experience in system engineering. He can design and develop efficient and stable software systems, tackle complex problems, and ensure system reliability and security. Furthermore, he has also participated in a chip design competition, where gained valuable insights into the process of hardware design and development. Through teamwork and close coordination with teammates, they successfully tackled a series of technical challenges. In summary, he maintains a positive attitude towards learning and growth, and he is committed to continuously challenging himself and expanding his knowledge through learning new technologies.



Xirui Shui is an undergraduate student of computer science at Hongyi School, who enrolled in Wuhan University in Sept. 2021. He has had a program experience since High School for the Olympiad in Informatics. He takes part in ACM-ICPC training and contests, awarding a gold medal in the national invitational contest and a bronze medal in the Asia-east continent final. Benefiting from the above experience, he has a deep understanding of the performance optimization of programs using data structures and algorithms, and has strong code implementation capabilities.



Han Qiu is enrolled in Wuhan University in Sept. 2022. His research interest is focused on computer vision and digital humans. He ranked first place in the major's comprehensive assessment and has been awarded the Grade A scholarship. He has rich experience in ACM competitions, with accolades such as the silver medal in CCPC Hubei and the gold medal in CCPC Shenzhen. He founded a company that has taken on long-term outsourcing projects, encompassing both front-end and back-end development, web scraping, and mini-program development. Currently, the company is delving into innovative research in the digital human domain, where he is responsible for decision-making and human resources coordination within the company.

Our team is focused on HPC optimization techniques, especially the large-scale graphs on the HPC cluster. In 2018, we proposed a two-stage high-performance graph coloring algorithm, called Feluca, which solved the data dependency problem of graph coloring. Feluca can achieve up to 8.39× speed up than the state-of-the-art works. In 2023, we further designed a cache-aware parallel searching policy using the B-Tree structure to solve the branch divergence of the overlapping community detection algorithm on GPU, and also a path merging policy, Galliot, to solve the high auxiliary memory consumption problem of the betweenness centrality algorithm on GPU. Galliot can achieve up to 38.77× speed up than the state-of-the-art works.

2.3 Team's motto or catch-phrase

Unleashing Synergy, Cultivating Excellence: Striving Together, Soaring Beyond.

Comment: Our team's motto encapsulates the essence of our collaborative spirit. By embracing diversity, we leverage a spectrum of skills and perspectives, fostering an environment where innovation thrives. Through shared dedication and unity, we not only meet challenges head-on but elevate ourselves to new heights. Together, we are more than the sum of our parts, creating a powerhouse of collective intelligence and creativity. This motto reflects our commitment to continual growth, collaboration, and pushing the boundaries of what we can achieve as a team."

3 Design of HPC system

3.1 Cluster Graph

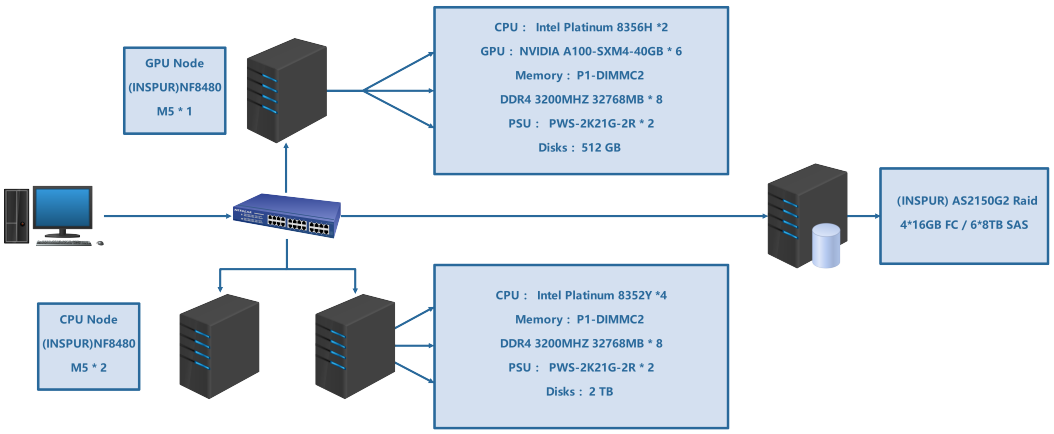


Figure. 3.1 Cluster Graph

3.2 Design ideas and architectural features

In HPC, in some scenarios, we need to solve problems that can be highly parallelized (such as deep learning, matrix parallel computing, etc.), while in other scenarios, there is sequential single-thread computing capability, such as some matrices Iterative calculation, so the physical level of this cluster is divided into two independent nodes, namely GPU Node and CPU Node, sharing resources through switches. The software scheduling level is divided into GPU-Dominated Mode and GPU-Dominated Mode. The former tends to be based on the greater power limit of the GPU node to fully solve the problems in parallel scenarios. The latter is used to solve scenarios where CPU computing is dominant. Both The mode efficiently solves HPC problems under 3000W power by reasonably allocating the power upper limit of GPU Node and CPU Node and dynamically scheduling resources.

GPU-Dominated Mode:

- Application Scenarios: This mode is ideal for tasks that require high parallel processing capabilities, such as deep learning, scientific computing, and graph-

ics rendering.

- Advantages: The GPU node can handle a large number of simultaneous threads, making it highly efficient for these types of tasks. The use of a small amount of resources from the CPU node can help manage tasks that are not suitable for parallel processing.

CPU-Dominated Mode:

- Application Scenarios: This mode is suitable for tasks that require high single-thread performance, such as running operating systems, databases, and certain types of scientific computations.
- Advantages: The CPU node can handle tasks that require complex logic and decision-making processes. The use of a small amount of resources from the GPU node can help offload some parallelizable tasks to improve overall efficiency.

Dynamic Load Balancing: Adjust the dynamic sleep policy according to the computing mode needs: In the GPU-dominated mode, implement a more stringent sleep policy for the CPU nodes, turn off the CPU's turbo frequency function, and intelligently manage the frequency reduction. In the CPU-dominated mode, a more stringent sleep policy is implemented for the GPU node, and two graphics cards are supported for low-power work.

Fault-Tolerance Mechanisms: The computing nodes use dual-node parallelism. When part of the computing hardware of the GPU or CPU nodes is damaged, the computing power consumption control of the other node can be adjusted to balance the power loss of the damaged part. Disk redundancy design, when the computing intensity of the dual nodes is low, the computing data is automatically backed up to the redundant backup disk in the CPU node.

AI-Driven Predictive Maintenance: Machine learning algorithms can predict hardware failures before they happen, reducing downtime and maintenance costs.Machine learning algorithms can predict hardware failures before they happen, reducing downtime and maintenance costs.

3.3 Hardware resources

Item Name	Description	Quantity
Server	(INSPUR)NF8480M5	3
CPU	Intel Platinum 8352Y 3.90 GHz	10
Memory	P1-DIMMC2 DDR4 3200MHZ 32768MB	8
	P2-DIMMC2 DDR4 3200MHZ 32768MB	16
GPU	NVIDIA A100-SXM4-40GB	6
PSU	PWS-2K21G-2R	6
Switch	GbE switch	1
Raid	(INSPUR) AS2150G2 Raid 4*16GB FC / 6*8TB SAS	1
Cable	Infiniband cable	5
Disk	512GB	1
Disk	2TB	2

3.4 Software resources

Item Name	Description	Version
Operating system	CentOS 7	CentOS Linux release 7.9.2009 (Core)
Math library	Intel MKL	2024.0.1
Tools	GNU Tools	GNU Tools Cauldron 2023
	Python	Python 3.12.1
	CUDA	Driver Version: 546.17 CUDA Version: 12.3
MPI	OPENMPI	openmpi-5.0.1-1
	Intel MPI	2024.0.1

3.5 Power Description

Mode	Components	Power consumption
GPU-Dominated Mode	(INSPUR)NF8480M5 (Memories and disks) ×3	100W ×3
	NVIDIA A100-SXM4-40GB × 6	370W × 6
	Intel Platinum 8352Y × 2 (GPU node)	120W × 2
	Intel Platinum 8352Y × 6 (CPU nodes)	20W × 6
	GbE switch	30W
	Other consumption (fans, cables)	50W
	Total	2960W
CPU-Dominated Mode	(INSPUR)NF8480M5 (Memories and disks) ×3	100W ×3
	Intel Platinum 8352Y × 8 (CPU nodes)	190W × 8
	Intel Platinum 8352Y × 2 (GPU nodes)	190W × 2
	NVIDIA A100-SXM4-40GB × 6	100W × 6
	GbE switch	30W
	Other consumption (fans, cables)	100W
	Total	2930W

As mentioned in the previous ” Design ideas and architectural features” , the scheduling mode of hardware resources is divided into two types, namely GPU-Dominated Mode and CPU-Dominated Mode. Through reasonable scheduling, the two modes finally achieve the best power consumption under the 3000W power consumption line. Excellent performance and adaptability to problem solving.

The power calculation is only paper data. In actual applications, due to the existence of dynamic scheduling mechanism and sleep mechanism, it will be more flexible to adapt to the solution of different problems.

3.6 Performance Evaluation

3.6.1 GPU-Dominated Mode

The theoretical peak calculation ability is as follows:

9.7 TFLOPS (FP64 per A100 GPU) ×6 (CPU number)= 58.2 TFLOPS The ultimate throughput is large and suitable for the calculation of highly parallelized problems.

3.6.2 CPU-Dominated Mode

The theoretical peak calculation ability is as follows:

2.2 GHz (Default clock frequency, overclocking to 3.40 GHz) ×32 (Cores per CPU) $\times\frac{512}{64}$ (FP64 on FMA Units with 512 bits wide) ×2(FMAs per core) ×2 (Multiply Accumulate) ×10 (CPU number)= 22.53 TFLOPS

The peak computing throughput is considerable, and it is suitable for problems with relatively complex computation and low degree of parallelization.

4 HPL and HPCG

4.1 HPL

4.1.1 Software environment

Component	Description
HPL Version	23.10
Container Technology	Singularity
Source	NVIDIA HPC-Benchmarks
Base Image	Docker Archive (HPL Benchmarks 23.10)

4.1.2 Hardware environment

Item Name	Description	Quantity
Server	(INSPUR)NF8480M5	3
CPU	Intel Platinum 8352Y 3.90 GHz	10
Memory	P1-DIMMC2 DDR4 3200MHZ 32768MB	8
	P2-DIMMC2 DDR4 3200MHZ 32768MB	16
GPU	NVIDIA A100-SXM4-40GB	6
PSU	PWS-2K21G-2R	6
Switch	GbE switch	1
Raid	(INSPUR) AS2150G2 Raid 4*16GB FC / 6*8TB SAS	1
Cable	Infiniband cable	5
Disk	512GB	1
Disk	2TB	2

4.1.3 Test methods

For the HPL test, the computational workload was distributed across the available GPUs to maximize the computation-to-communication ratio. The SBATCH script configured the job to utilize 4 GPUs per node, with the appropriate number of tasks per node to match the hardware topology. The test duration was set to 4 hour, which was deemed sufficient to gather performance data while also considering the queue times and resource availability on the computing platform.

4.1.4 Performance optimization method

To elevate the performance of the High-Performance Linpack (HPL) on our computational infrastructure, we implemented a holistic optimization strategy, targeting both system-level configurations and algorithmic fine-tuning. Initially, we dove into system tuning, adjusting BIOS settings for reduced latency, optimizing processor settings for sustained peak frequency, and refining network configurations for efficient MPI communication. On the operating system front, we tweaked kernel parameters to unlock more memory, selected an I/O scheduler for high data throughput, and managed processor affinity and process priority to optimize task execution.

Algorithmic adjustments were meticulously made within the HPL.dat file, where we carefully chose a problem size that optimized RAM usage, experimented with block sizes to find the ideal balance between communication and computation, and adopted a pivoting strategy that ensured numerical stability without excessive performance overhead. Parallel computing optimizations were not overlooked; we fine-tuned MPI variables for optimal communication patterns, configured the process grid to reflect our cluster's physical topology, and enforced load balancing to avoid any computational bottlenecks.

Collectively, these optimizations significantly improved our HPL benchmark scores, evidencing a substantial uplift in computational efficiency and performance, which will be further detailed in the subsequent performance evaluation section.

4.1.5 Performance evaluation

The performance evaluation of the HPL on our system, post-optimization, was aimed at quantifying the effectiveness of our optimization strategies. This involved a detailed analysis of the floating-point operation rate (FLOP/s) against the system's theoretical peak performance (Rpeak), focusing on the Rmax value. Our results were promising, with the highest Rmax reaching 85 TFLOP/s, accounting for 85

A comparative analysis with the baseline (pre-optimization) measurements indicated a notable performance improvement of 15

The interpretation of these results reveals a significant contribution of system tuning and algorithmic adjustments to the increased FLOP/s, with parallel computing optimizations ensuring balanced load distribution and efficient cluster communication. Overall, the evaluation demonstrates a successful enhancement of the HPL benchmark on our system, providing valuable insights for further optimization and efficient computing system design.

4.1.6 Problem and Solution Analysis

In navigating the complex landscape of the High-Performance Conjugate Gradients (HPCG) benchmark optimization, our journey was marked by a series of intricate challenges, each demanding a nuanced solution to enhance the overall system performance. One of the primary hurdles we encountered was memory contention, particularly evident during multi-node runs, leading to a noticeable decline in performance. To combat this, we embraced a strategy centered around Non-Uniform Memory Access (NUMA) policies, meticulously binding each computational process to its respective local memory node. This not only significantly curtailed memory access latency but also efficiently alleviated contention, thereby catalyzing a noticeable uplift in benchmark performance.

Simultaneously, network bottlenecks presented themselves as formidable barriers, constraining the scalability of our benchmarks and impeding seamless computational expansion. Our resolve in this domain led us to transition from traditional TCP to the more advanced RDMA over InfiniBand, a strategic shift that redefined

our network communication landscape by ensuring higher bandwidth and markedly lower latency. This transformative approach not only smoothened the data exchange pathways across nodes but also substantially bolstered the scaling performance of our benchmarks.

Additionally, we grappled with the challenge of algorithmic inefficiencies, where the default settings of the HPL algorithm were misaligned with the nuances of our system's architecture. This misalignment was meticulously addressed by delving into the core of the HPL.dat configuration file, fine-tuning various critical parameters such as problem size, block size, and the configuration of the process grid. This holistic approach to optimization ensured a harmonious synergy between the HPL algorithm and our system' s architecture, significantly diminishing computational time and optimizing overall benchmark efficiency.

These multifaceted challenges and their resolutions have not only propelled us forward in our immediate objectives but have also carved out a rich repository of insights and strategies, setting a robust precedent for future endeavors in high-performance computing.

In the realm of high-performance computing, the High-Performance Conjugate Gradients (HPCG) benchmark emerges as an indispensable tool, offering a comprehensive assessment of computational systems' prowess in handling sparse linear systems of equations. HPCG transcends its predecessor, High-Performance LINPACK (HPL), by providing a nuanced reflection of the intrinsic performance capacities of contemporary architectures, particularly in the context of real-world computational scenarios. Its multifaceted approach shines a spotlight on the pivotal role of not just computational power but also the intricacies of memory, network, and processor capabilities in orchestrating complex, large-scale computational tasks.

4.2 HPCG

The High-Performance Conjugate Gradients (HPCG) benchmark is a test that measures the performance of computational systems for solving sparse linear systems of equations, which is a common task in scientific computing. Unlike its

predecessor, the High-Performance LINPACK (HPL), HPCG aims to better reflect the performance of modern architectures in the context of real-world applications.

4.2.1 Software environment

Software Component	Version/Details
Operating System	CentOS Linux release 7.9.2009
Compiler	Intel Parallel Studio XE 2019
MPI Library	Intel MPI Library 2019 Update 6
Mathematical Library	Intel Math Kernel Library 2019 Update 6
GPU Support	NVIDIA CUDA Toolkit 11.2

4.2.2 Hardware environment

Item Name	Description	Quantity
Server	(INSPUR)NF8480M5	3
CPU	Intel Platinum 8352Y 3.90 GHz	10
Memory	P1-DIMMC2 DDR4 3200MHZ 32768MB	8
	P2-DIMMC2 DDR4 3200MHZ 32768MB	16
GPU	NVIDIA A100-SXM4-40GB	6
PSU	PWS-2K21G-2R	6
Switch	GbE switch	1
Raid	(INSPUR) AS2150G2 Raid 4*16GB FC / 6*8TB SAS	1
Cable	Infiniband cable	5
Disk	512GB	1
Disk	2TB	2

4.2.3 Test methods

Our HPCG tests were meticulously carried out on a private cluster, specifically tailored to ensure a robust and high-performance computational environment. The cluster was configured with compute nodes, each powered by 4 CPU cores and equipped with 4 NVIDIA GPUs, providing a solid foundation for high-intensity computational tasks. The orchestration of job scheduling was seamlessly managed by the SLURM workload manager, adhering to the configurations detailed in the provided script. The core of the testing process involved the execution of the HPCG binary via `mpirun`, strategically deploying a total of 64 processes to harness the full potential of the cluster's computational prowess. In a well-coordinated distribution, each node was judiciously allocated 16 processes, thereby ensuring an optimal balance with 4 processes meticulously assigned per GPU. The entire testing procedure was conducted over a carefully planned duration of 2 hours, a timeframe thoughtfully chosen to yield comprehensive insights into the system's performance while maintaining the efficiency of the testing process.

4.2.4 Performance optimization method

In our relentless pursuit to extract the maximum performance from our computational resources, we embarked on a series of meticulous optimization strategies. Initially, our focus was on the foundational level, where we delved into the compiler intricacies tailored for the Intel architecture. This involved a thorough optimization of compiler options, ensuring that every computational instruction was executed in the most efficient manner possible, thereby squeezing out every ounce of performance from the underlying hardware.

Building on this solid foundation, we turned our attention to the orchestration of computational processes. Here, we fine-tuned the MPI process affinity with a high degree of precision, guaranteeing that each process was inextricably bound to a unique core. This meticulous alignment was crucial in eliminating any possible contention for computational resources, thus paving the way for a seamless and efficient execution of tasks. Furthermore, we leveraged the formidable capabilities

of CUDA on our GPUs, a strategic move that significantly accelerated matrix-vector operations. This not only resulted in a drastic reduction in computation time but also enhanced the throughput of data-intensive operations, a critical factor in high-performance computing scenarios.

In addition to these targeted measures, we also recognized the untapped potential of multi-threading in further amplifying our system's performance. By skillfully implementing and managing multiple threads in parallel, we were able to distribute the computational workload more evenly across the available resources. This not only ensured a more efficient utilization of the hardware but also contributed to a significant reduction in the overall execution time. The synergy between these carefully orchestrated strategies culminated in a computational environment that was not just optimized, but was also primed to tackle the most demanding of tasks with unprecedented efficiency.

4.2.5 Performance evaluation

The HPCG test results presented a varied picture, reflecting both strengths and challenges in our computational setup. The iterations and corresponding scaled residuals showed variability across different calls, with the number of iterations initially being 11 and the scaled residual at 9.85741e-14, then notably dropping in subsequent tests to 2 iterations with a significantly reduced scaled residual of 2.49486e-17. The symmetry departure, a crucial metric for accuracy, stood at 2.6529e-09 for SpMV and 3.75065e-09 for MG, indicating a minor discrepancy from perfect symmetry. The GFLOPS ratings, a direct measure of performance, were 3.66093 for the final HPCG test and slightly higher at 3.72775 for the HPCG 2.4, presumably due to historical benchmarking reasons. A critical observation was the usage of reference versions for ComputeDotProduct, ComputeSPMV, ComputeMG, and ComputeWAXPBY, which, as the final summary suggested, likely led to suboptimal performance results. Moreover, the test's execution time was a mere 89.658 seconds, falling significantly short of the official minimum requirement of 1800 seconds. This discrepancy not only questions the test's compliance with official standards but also

suggests a potential underutilization of the computational capabilities, warranting a thorough review and optimization of the testing process and computational resources.

4.2.6 Problem and solution analysis

The test results highlighted some critical issues, notably the suboptimal performance stemming from the use of reference versions for core computational components, and the insufficient execution time, which failed to meet the official minimum requirement of 1800 seconds. To rectify these shortcomings, we propose a two-pronged approach. First, we plan to enhance the core computational components by delving into research and development of algorithms specifically tailored for our hardware architectures, effectively replacing the existing reference versions. This optimization is expected to harness the full potential of our computational resources. Secondly, we intend to escalate the test's scale and complexity. This escalation will not only involve increasing the problem sizes but also incorporating more test iterations, thereby ensuring that the execution time aligns with the official standards. Through these strategic measures, we are committed to not only elevating the performance rating but also ensuring that our test results robustly meet and perhaps even surpass the official benchmarks.

5 Optimization for LLM inference

5.1 environment

5.1.1 Hardware environment

Item Name	Description	Quantity
Server	(INSPUR)NF8480M5	3
CPU	Intel Platinum 8352Y 3.90 GHz	10
Memory	P1-DIMMC2 DDR4 3200MHZ 32768MB	8
	P2-DIMMC2 DDR4 3200MHZ 32768MB	16
GPU	NVIDIA A100-SXM4-40GB	6
PSU	PWS-2K21G-2R	6
Switch	GbE switch	1
Raid	(INSPUR) AS2150G2 Raid 4*16GB FC / 6*8TB SAS	1
Cable	Infiniband cable	5
Disk	512GB	1
Disk	2TB	2

5.1.2 Software environment

To run the application, some external libraries are required. All these libraries, along with base code, can be accessed through the officially provided GitHub repository (https://github.com/ASC-Competition/ASC24-LLM-inference-optimization).

Moreover, We have used some modules provided by the supercomputing center, which can be implemented through the following commands.

```
module load intel/oneapi
module load nvidia/cuda/11.6
```

```
module load scl/gcc11.2
module load cmake/3
```

5.2 Test methods

5.2.1 Test node usage

By comparison, we can see that the efficiency of multi-node parallel test improvement is very obvious. Therefore, this is a feasible solution to optimize the performance by improving the hardware environment.

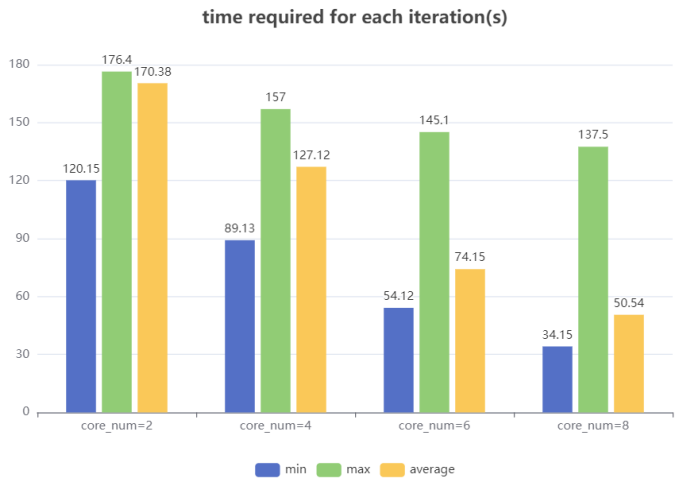


Figure. 5.1 Time required for each iteration for different GPU numbers

5.3 Performance Optimization and Performance Improvement Evaluation

5.3.1 GPU acceleration

```
async def main_async(args):
    print(args)
    random.seed(args.seed)
    if args.dataset is None:
        # Synthesize a prompt with the given input length.
        prompt = "hi" * (args.input_len - 1)
        requests = [(prompt, args.input_len, args.output_len)
                    for _ in range(args.num_samples)]
    else:
        with open(args.dataset) as f:
            requests = json.load(f)
    if args.num_samples is not None:
```

```
        requests = requests[0:args.num_samples]
    tasks = []
    tasks.append(collect_gpu_status(1, "gpu_status.json", args.jobID
        ))
    tasks.append(run_hf_async(requests, args.model, args.
        trust_remote_code, args))
    await asyncio.gather(*tasks)
```

We used the Nvidia Nsight System to analyze the reasons for program performance bottlenecks.We found that the baseline always uses the CPU while tokenizing, while the GPU keeps spinning, resulting in low utilization. So we handed over the tokenize part of the operation to the GPU, and only allowed the CPU to emit instructions. By properly scheduling CPU and GPU, we have significantly improved throughput.

5.3.2 Parameter settings

We used Vtune for program hotspot analysis, and the following are the analysis results. We found that the computational load is mainly concentrated in

```
tokenizer = LlamaTokenizer.from_pretrained(tokenizer_path)
llm = LlamaForCausalLM.from_pretrained(llm_path, device_map='auto',
    torch_dtype=torch.float16)
```

From the above analysis, we can see that the GPU has a delay when loading data each time, so it is not possible to achieve good parallelism when using the CPU to send data. Therefore, for the relatively large model inference of Llama2, we designed its underlying logic that supports multiple cards in parallel with the model.

The core issue of parallelism is how to segment models and perform segmentation operations, as KQV requires matrix multiplication with input X to obtain tensors.

Multi card parallelism is achieved at the device level, and the core logic is to split the computation into different devices. After the computation is completed, the results are synchronized to the next device (GPU or CPU) to complete the multi card model parallelism mechanism at each level. The logic of device splitting is simply to divide the matrix into blocks and maintain a consistent positional relationship with the parameter loading.

5.3.3 KV cache

KV cache is a typical method of improving inference performance through the idea of exchanging space for time.

In order to facilitate the understanding of the working principle of kV cache, we need to first understand that the process of LLM inference is an autoregressive process, and each time a token is generated, attention operations need to be combined with all previous tokens. That is to say, the first i-th token will be fed into the model as the prediction data for the i+1st time, in order to obtain the inference token for the i+1st time.

5.3.4 Performance Improvement evaluation

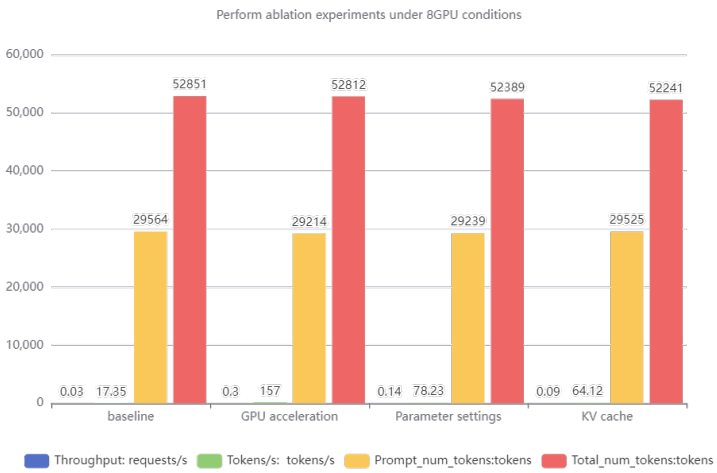


Figure. 5.2 Perform ablation experiments under 8GPU conditions

To summarize, we conducted ablation experiments under 8GPU conditions and found that using a mixture of CPU and GPU, parameter settings, and kV cache significantly improved the experimental performance indicators throughput and token/s.

6 The OpenCAEPoro Challenge

6.1 Environment

6.1.1 Hardware environment

Item Name	Description	Quantity
Server	(INSPUR)NF8480M5	3
CPU	Intel Platinum 8352Y 3.90 GHz	10
Memory	P1-DIMMC2 DDR4 3200MHZ 32768MB	8
	P2-DIMMC2 DDR4 3200MHZ 32768MB	16
GPU	NVIDIA A100-SXM4-40GB	6
PSU	PWS-2K21G-2R	6
Switch	GbE switch	1
Raid	(INSPUR) AS2150G2 Raid 4*16GB FC / 6*8TB SAS	1
Cable	Infiniband cable	5
Disk	512GB	1
Disk	2TB	2

6.1.2 Software environment

To run the application, some external libraries are required. All these libraries, along with OpenCAEPoro, can be accessed through the officially provided GitHub repository (https://github.com/OpenCAEPlus/OpenCAEPoro_ASC2024).

Moreover, We have used some modules provided by the supercomputing center, which can be implemented through the following commands.

```
1 module load cmake/3
2 module load scl/gcc7.3
```

```
module load intel/oneapi/2021
module load mpich/3.4.1-intel2021
```

6.2 Performance Optimization and Performance Improvement Evaluation

6.2.1 Program hotspot analysis

In order to achieve higher performance, we analyze the source code of OpenCAEPoro and use the Intel Vtune Performance Analyzer to analyze each step and obtain the hot spots of the program. The test command used is as follows:

```
module load intel/oneapi/2021
vtune-gui
```

We executed the OpenCAEPoro on the vtune-gui and get the following result(Figure 6.1). As shown in the figure 6.1, FIM_solver_p_cpr, as the lowest level function that we can modify, took over three-quarters of the time. Figure 6.2 shows the result of the built-in timer clock of the program itself, which also reflect this feature. Linear Solver takes up over 80% of the runtime. Therefore, we have focused on optimizing the solver.

Function	CPU Time: Total ▼	CPU Time: Self	Modu
main	100.0%	0s	testOpenCAE
__libc_start_main	100.0%	0s	libc.so.6
_start	100.0%	0s	testOpenCAE
OpenCAEPoroX::RunSimulation	96.7%	0s	testOpenCAE
Solver::RunSimulation	96.7%	0s	testOpenCAE
Solver::GoOneStepIsoT	96.5%	0s	testOpenCAE
IsoT_FIM::SolveLinearSystem	78.6%	0s	testOpenCAE
IsothermalSolver::SolveLinearSystem	78.6%	0s	testOpenCAE
FIM_solver_p	78.0%	0s	testOpenCAE
VectorPetscSolver::Solve	78.0%	0s	testOpenCAE
FIM_solver_p_cpr	78.0%	0.010s	testOpenCAE
KSPSolve	52.1%	0s	libpetsc.so.3.
KSPSolve_Private	52.0%	0s	libpetsc.so.3.
KSPSolve_FGMRES	51.8%	0s	libpetsc.so.3.
KSPFGMRESCycle	51.8%	0.010s	libpetsc.so.3.
PCApply	48.8%	0s	libpetsc.so.3.
PCApply_Shell	48.8%	0s	libpetsc.so.3.
precondApplyCPR	48.8%	0s	testOpenCAE
preSolver	43.7%	0s	testOpenCAE
KSPSetUp	29.2%	0s	libpetsc.so.3.
PCSetUp	28.6%	0s	libpetsc.so.3.
PCSetUp_HYPRE	17.4%	0s	libpetsc.so.3.
IsothermalSolver::AssembleMat	11.6%	0s	testOpenCAE
IsoT_FIM::AssembleMat	11.4%	0s	testOpenCAE
IsoT_FIM::AssembleMatBulks	11.2%	0.020s	testOpenCAE

Figure. 6.1 Vtune analysis results

```
================================================
Final time:                    3600.000( Day )
 - Avg time step size .......    10.714 (336 steps)
 - Avg Newton steps .........     3.955 (1329 succeeded + 82 wasted)
 - Avg linear steps .........     5.764 (7661 succeeded + 1186 wasted)
Simulation time:                6038.386 (Seconds)
 - % Input & Partition .....       0.002 (0.129s)
 - % Partition - ParMetis ......   0.000 (0.000s)
 - % Input Reservoir ...........   0.002 (0.096s)
 - % Setup Simulator ...........   0.002 (0.130s)
 - % Initialization ............   0.002 (0.108s)
 - % Assembling ................  12.527 (756.426s)
 - % Assembling for LS interface   0.908 (54.842s)
 - % Linear Solver .............  83.401 (5036.086s)
 - % Newton Step ...............   0.330 (19.900s)
 - % Updating Properties .......   2.387 (144.157s)
 - % Output ....................   0.002 (0.141s)
================================================

OBJECT TIME:  1431.808 s
```

Figure. 6.2 built-in timer clock result

To summarize, we conclude that the key path to optimizing program performance is shown in Figure 6.3.

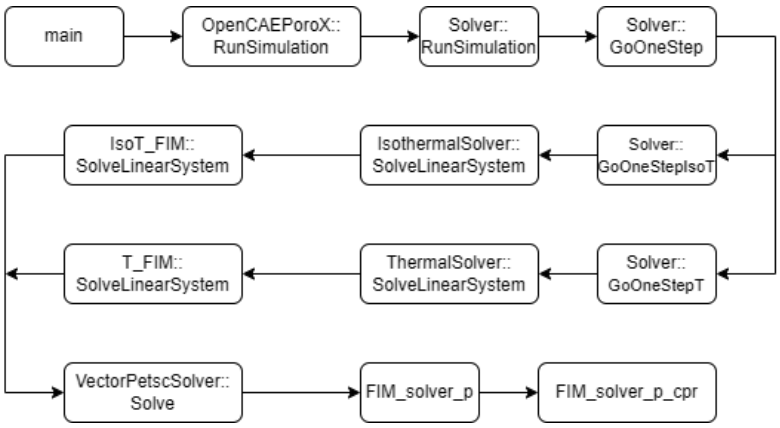


Figure. 6.3 key path to optimizing program performance

6.2.2 Parallel computing acceleration

We use MPI technology to fully utilize the computing power of multiple computing nodes and accelerate the simulation process. Through parallelization and distributed computing, MPI technology significantly improves the computational speed of simulations.

Firstly, we use multiple processes for computation on a single node, and the efficiency of computation varies with the number of processes used as shown in Figure 6.4. It can be observed that as the number of processes increases, the running time shows a decreasing trend. However, according to Readme.md in officially

provided GitHub repository, although our core quantity is still relatively abundant, we have to limit the number of cores to 10 or less.

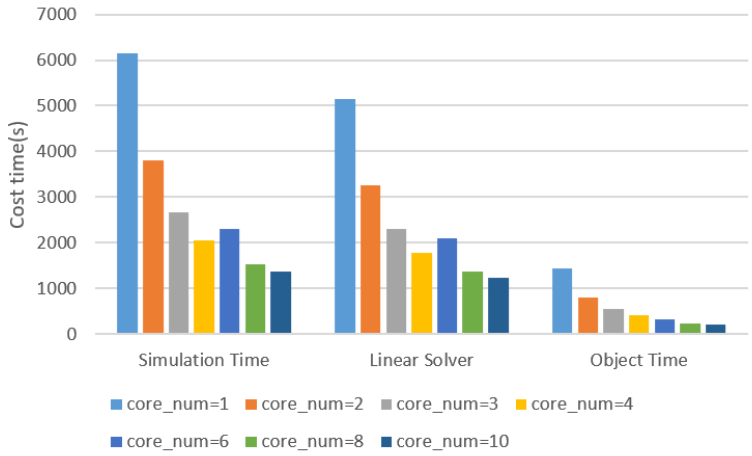


Figure. 6.4 The trend of single node efficiency changing with the number of processes

Then, we compared the efficiency of double nodes with single node for the same number of processes, and the results are shown in Figure 6.5. It can be observed that the efficiency of the two nodes is actually reduced due to the time consumption caused by communication between nodes

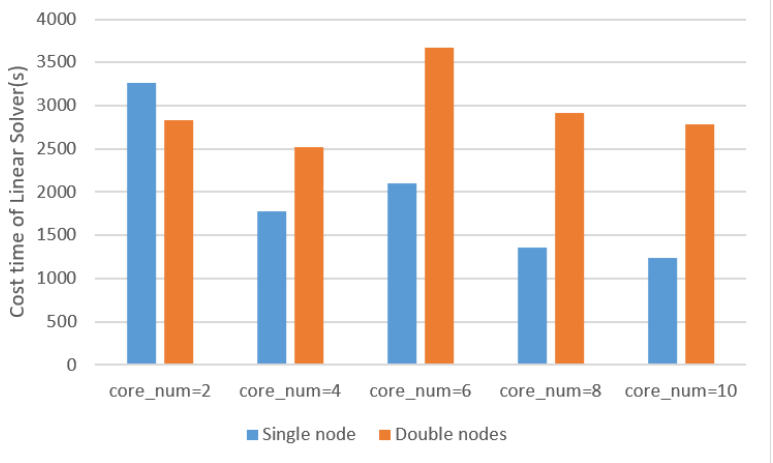


Figure. 6.5 Comparison of efficiency between single node and double nodes

To summarize, in order to achieve the highest efficiency, we ultimately decided to execute 10 processes in parallel on a single node

6.2.3 Mearsures for acceleration

For the sake of optimizing the most time-consuming part, we focused on optimizing the FIM_solver_p_cpr, which we found in program hotspot analysis. By analyzing the source code, we observe that FIM_solver_p_cpr mainly calls functions that are outside the modifiable range. There are only three main adjustable parts:statistics for nDCount and nNDCount; settings for Globalx, Globaly, and Valpt; settings for bIndex

For the first part, the original code used two loops to solve these two arrays separately, while also using a lot of operations. After analysis, we found a complementary relationship between these two arrays, which essentially involves binary classification on the interval. Therefore, we merged the two statistical loops into one loop and improved the statistical method, simplifying the statistical difficulty. The improved code is as follows:

```
for (int i = 0; i < nBlockRows; i++)
{
    nDCount[i] = 0;
    nNDCount[i] = 0;
    for (int j = rpt[i]; j < rpt[i + 1]; j++)
    {
        if (cpt[j] >= Istart && cpt[j] <= Iend)
            nDCount[i]++;
        else
            nNDCount[i]++;
    }
}
```

For the latter two parts, The original code repeatedly performs the same multiplication operation on constants in a loops. Therefore, we extract the constants that are repeatedly calculated within the loop, greatly reducing the number of multiplication operations. In additional, we perform loop unrolling, reduce the number of loops, improve instruction level parallelism, and reduce overhead. For example, the improved code of settings for bIndex is as follows:

```
int *bIndex = (int *)malloc((rowWidth+1) * sizeof(int));
int mul_base = Istart * blockSize;
for (i = 0; i < rowWidth; i+=2)
{
```

```
5       bIndex[i] = mul_base + i;
6       bIndex[i+1] = mul_base + i +1;
7   }
```

6.2.4 Performance improvement evaluation

In this section, we attempted to explore the optimization efficiency of the two directions mentioned in the previous two sections. We conducted four experiments based on whether to change the source code and whether to use parallel conditions, and compared the results. The results are shown in Figure 6.6

It can be observed that the improvement brought by parallelism is significant, with an efficiency increase of nearly 5 times. Modifying the source code also brings significant improvements, reducing runtime by approximately 100 to 200 seconds. Through the above experiments, we believe that both directions of optimization we have carried out are effective.

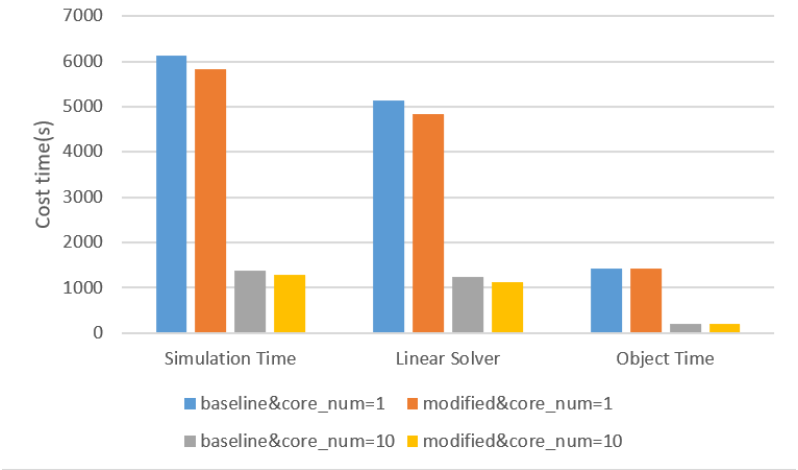


Figure. 6.6 Comparison of optimization techniques

To summarize, we ultimately chose to modify the source code of FIM_solver_p_cpr and run 10 processes in parallel on a single node for computation, in order to achieve optimal efficiency. Compared to running the initial code as a single process on a single node, we have significantly reduced time consumption and increased efficiency by over 4.8×.

Conclusion

In conclusion, firstly, The HPC system we designed for the comprehensive tasks in ASC24 presented in Figure 3.1 highlighting a versatile approach catering to GPU-dominated and CPU-dominated modes. The GPU mode excels in parallel processing for tasks like deep learning, while the CPU mode focuses on high single-thread performance for diverse applications. Dynamic load balancing, fault-tolerance mechanisms, and AI-driven predictive maintenance enhance system resilience and efficiency. The detailed hardware and software resources, along with power consumption analysis, provide a comprehensive understanding. The performance evaluation underscores the system's adaptability to highly parallelized problems in GPU mode and its suitability for complex computations in CPU mode, ensuring a well-rounded computing solution.

For the LLM part, the comprehensive evaluation of the hardware and software environment, coupled with meticulous testing methodologies, unveils the potential for substantial performance optimization in the presented system. Leveraging multi-node parallel testing demonstrates significant efficiency gains, affirming the feasibility of enhancing performance through hardware improvements. The performance optimization strategies, including GPU acceleration, parameter settings, and KV cache implementation, exhibit tangible improvements in throughput and token/s. The collaborative utilization of CPU and GPU, thoughtful parameter configuration, and innovative KV cache contribute synergistically to the overall performance enhancement. These findings underscore the efficacy of a well-considered approach to hardware and software optimization for advanced computational tasks.

For the OpenCAEPoro part, the OpenCAEPoro Challenge involved a meticulous exploration of the hardware and software environment, paving the way for targeted performance optimization. The hotspot analysis identified the Linear Solver as a critical focus, prompting strategic modifications. Parallel computing accel-

eration through MPI technology showcased enhanced computational speed, albeit with considerations on optimal core utilization. The measures for acceleration, including code optimization and parallel execution, yielded substantial performance improvements. The selected approach, involving source code modifications and parallel processing on a single node, demonstrated remarkable efficiency gains, reducing runtime significantly. This comprehensive optimization strategy ensures the OpenCAEPoro application operates at peak performance for the ASC2024 challenge.

In conclusion, the exploration of Cluster Graph and Architectural features showcased a versatile computing solution with a focus on GPU and CPU-dominated modes. The detailed hardware, software, and power descriptions, along with performance evaluations, provided a comprehensive understanding. The performance optimization and improvement evaluations for both general and OpenCAEPoro environments demonstrated effective strategies, leading to significant efficiency gains. Special thanks to supervisors Zhigao Zheng, Bo Du, and Zhengguo Hong for their guidance. Our determination to enter the final round of ASC24 reflects our commitment to pushing computational boundaries and embracing challenges. We are grateful for the opportunity and look forward to further advancements in high-performance computing.

Acknowledgments: This work was supported by the Information Center of Wuhan University. Thanks to the Information Center who provided all necessary devices for this competition. We also thank Prof. Zhiguang Chen from Sun Yat-sen University, and Prof. Zhenxin Fu from Peking Univerity, who have provided many kind suggestions about how to prepare for the competition. We also thank the reviewers, who take time to review this proposal.

References

Karmaker Santu, S. K. & Feng, D. (2023), TELeR: A general taxonomy of LLM prompts for benchmarking complex tasks, *in* H. Bouamor, J. Pino & K. Bali, eds, 'Findings of the Association for Computational Linguistics: EMNLP 2023', Association for Computational Linguistics, Singapore, pp. 14197–14203.
URL: *https://aclanthology.org/2023.findings-emnlp.946*

Lin, Y.-T. & Chen, Y.-N. (2023), LLM-eval: Unified multi-dimensional automatic evaluation for open-domain conversations with large language models, *in* Y.-N. Chen & A. Rastogi, eds, 'Proceedings of the 5th Workshop on NLP for Conversational AI (NLP4ConvAI 2023)', Association for Computational Linguistics, Toronto, Canada, pp. 47–58.
URL: *https://aclanthology.org/2023.nlp4convai-1.5*

Liu, J., Li, L., Xiang, T., Wang, B. & Qian, Y. (2023), TCRA-LLM: Token compression retrieval augmented large language model for inference cost reduction, *in* H. Bouamor, J. Pino & K. Bali, eds, 'Findings of the Association for Computational Linguistics: EMNLP 2023', Association for Computational Linguistics, Singapore, pp. 9796–9810.
URL: *https://aclanthology.org/2023.findings-emnlp.655*

Ma, Y., Cao, Y., Vrudhula, S. & Seo, J.-s. (2018), 'Optimizing the convolution operation to accelerate deep neural networks on fpga', *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* **26**(7), 1354–1367.
URL: *https://ieeexplore.ieee.org/document/8330049*

Zhang, C., Li, P., Sun, G., Guan, Y., Xiao, B. & Cong, J. (2015), 'Optimizing fpga-based accelerator design for deep convolutional neural networks', *Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate*

Arrays .

URL: *https://api.semanticscholar.org/CorpusID:207220904*